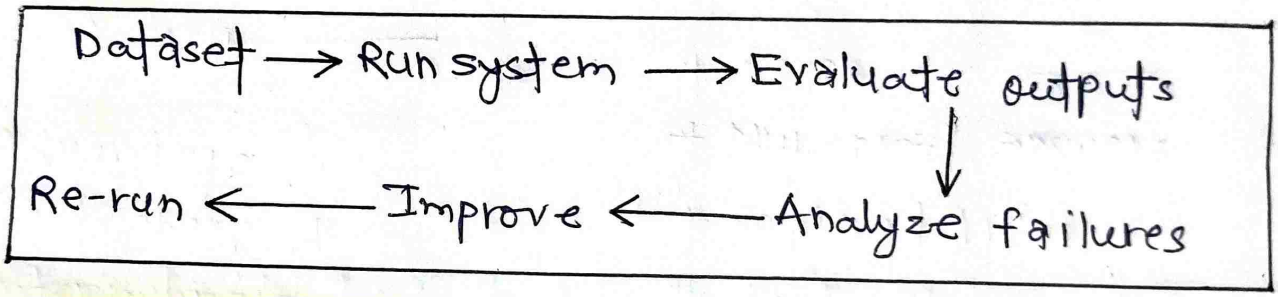
EVALS-IT
(Evaluating LLM Applications)

→ **LLM Evaluation (Evals)** - It's a systematic & repeatable way to test whether an LLM or LLM application is performing correctly according to predefined criteria.

↳ **Flow**



↳ **3 properties of a good Evals**

- systematic → structured test cases
- Repeatable → same test can be run after every change.
- Measurable → Defined criteria / metrics

→ **What can we evaluate ?**

An AI application can have multiple evaluation layers.

↳ **LLM**

- correctness
- Relevance
- Instruction following
- safety
- Reasoning

↳ RAG

- Retrieval quality
- context relevance
- Faithfulness
- Answer correctness

↳ Agent

- Tool selection
- Tool arguments
- Task completion
- No. of unnecessary steps
- Final answer quality

↳ Production

- Latency
- cost
- Error rate
- Reliability
- User satisfaction

→ Model Eval Vs Application Eval

Model Evaluation

Tests the base LLM itself

↳ Examples

- Reasoning
- Mathematics
- coding
- Knowledge
- Instruction following
- Long-context understanding
- Multimodal capabilities
- Tool calling

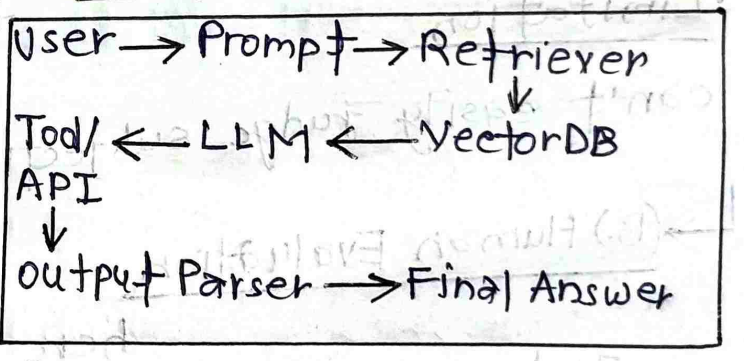
↳ Typing benchmark

capability	Example benchmark
• Knowledge / reasoning	MMLU
• Mathematics	GSM8K
• coding	HumanEval / SWE-bench

Application Evaluation

Tests your actual AI system

↳ Example:



We evaluate whether the whole system actually works.

Capability	Example benchmarks
• Instruction following	IF Eval
• Long context	Needle in a Haystack
• Multimodal	MMMU

→ Evaluation Methods

There are 3 major Approaches:

↳ (A.) code-based Evaluation

use deterministic code.

• Example

```
Expected = "Billing"
Actual = "Billing"
→PASS
```

• Useful

* Exact matches
* JSON schema validation
* classification
* Regex
* Numerical answers
* Required fields
* Tool names
* API parameters

• Advantage

Fast, cheap & Deterministic

: Limitation

can't easily judge subjective/semantic answers.

↳ (B.) Human Evaluation

here we (Humans) manually judge outputs.

Useful when quality is difficult to define programmatically.

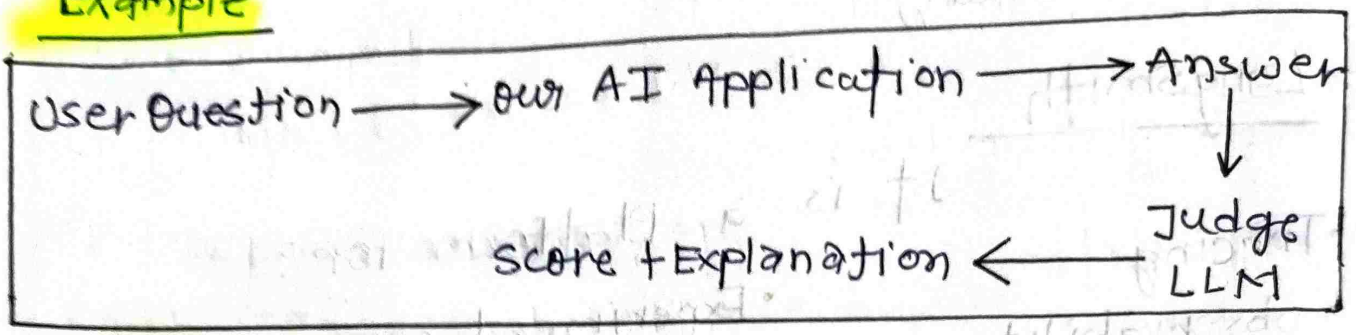
• Problems

* Expensive
* slow
* subjective
* Difficult to scale

(c) LLM-as-a Judge

Here another LLM to evaluate the output.

- **Example**



- **Example criteria**
  - correctness : 1-5
  - Relevance : 1-5
  - Faithfulness : 1-5
  - style : 1-5

- **Useful for**
  - open minded answers
  - semantic similarity
  - Helfulness
  - Tone
  - Relevance
  - RAG answer Quality

Note: The judge itself must be evaluated. A stronger/different model may sometimes provide more reliable judgements, but LLM-as-a-judge is not automatically ground truth.

RAG evaluation workflow

RAG evaluation workflow consists of three main steps:

(1) creating a dataset with questions & their expected answers

(2) Running our RAG Application on those questions.

(3) Using evaluators to measure how well our Application performed, looking at factor like:

- Answer relevance
- Answer Accuracy
- Retrieval Quality

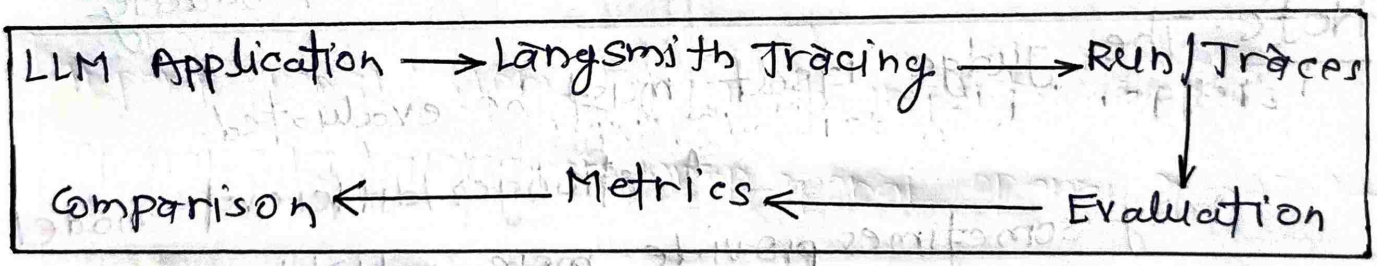
→ **LangSmith**

It is a Platform for:

- Tracing
- observability
- Testing
- Evaluation
- Experiment comparison
- Debugging LLM Applications

"Langsmith" is like as a development + Evaluation + observability platform for LLM applications.

↳ **Basic Flow**



→ **LLM chatbot Evaluation**

Tests an LLM chatbot using a dataset & evaluators to measure answer quality.

- setup → Load API keys + tracing
- Dataset → Questions + expected answers.
- chatbot → Generates answers
- Evaluators → check correctness + concision
- Evaluate → Run tests & collect results.

1. ENVIROMENT SETUP

In[1]:

```
import os
from dotenv import load_dotenv
load_dotenv() #Load variables from .env file

os.enivon ["LANGISMITH_API_KEY"] = os.getenv
("LANGISMITH_API_KEY")
os.environ["OPEN_API_KEY"] = os.getenv (
"OPEN_API_KEY")
#Load API Keys into enviroment variables

os.environ["LANGISMITH_TRACING" ] ="true"
#Enable Langsmith tracing
```

Take API's keys of Langsmith & openAI while login in there website.

2. CREATE LANGSMITH CLIENT & CREATE EVALUATION DATASET

In[2]:

```
#CREATE LANGSMITH CLIENT
from langsmith import client

client = Client()
#create client to interact with Langsmith
#Client() lets Python create datasets, run evaluations
 & access Langsmith functionality.

#CREATE EVALUATION DATASET
dataset_name = "chatbots Evaluation"
dataset = client.create_dataset(dataset_name)
#Create an empty dataset in Langsmith
```

3. ADD TEST CASES

In [8]

```
client.create_examples(dataset_id = dataset.id,
examples = [{"input": {"question": "What is langchain?"},
  "outputs": {"answer": "A framework for building
   LLM applications"},
   .
   .
   .
   ])
# Add Questions & their expected/reference answers
```

Each example has:
Inputs → user's question
Outputs → expected/reference answer

4. LANG-SMITH WRAPPER & DEFINE JUDGE INSTRUCTION

In [4]:

```
import openai from langsmith import wrappers
openai_client = wrappers.wrap_openai(openai.OpenAI())
#create openAI client
# wrapped client allows Langsmith to trace the calls
eval_instructions = """you are an expert professor
specialized in grading student's answer to question.
"""
#Instructions given to the LLM evaluator
```

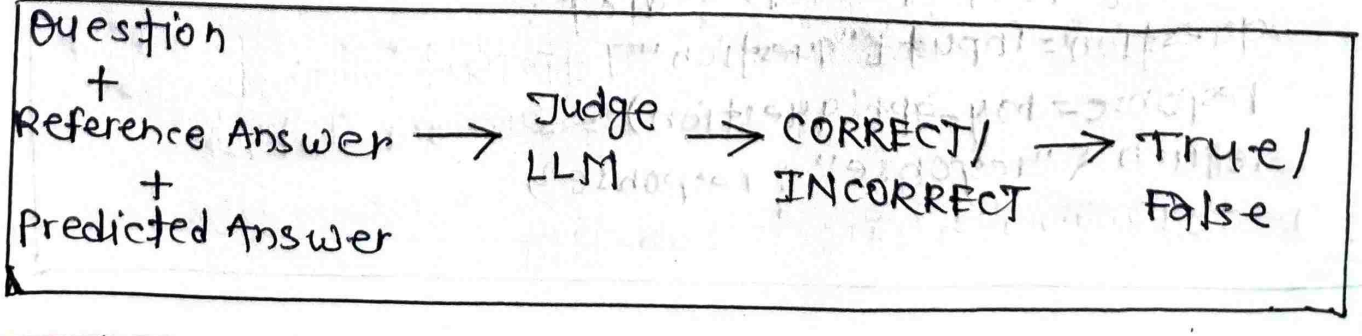
*Wrappers: It adds extra functionality around an existing object/function without changing its core behaviour.

5. CREATE CORRECTNESS EVALUATOR

In [5]:

```
def correctness(
    inputs: dict,
    outputs: dict,
    reference_outputs: dict ) -> bool:
    user_content = f"""" "you are grading the
    following question: {inputs['question']}
    Here is the answer: {reference_outputs[
    'answer']}
    you are grading the following predicted
    answer: {outputs['response']}
    Respond with CORRECT or INCORRECT
    Grade:
    """
    #Build the Grading prompt
    response = openai_client.chat.completions.create(
    model = "gpt-4o-mini", temperature = 0,
    messages = [{"role": "system",
    "content": eval_instructions},
    {"role": "user",
    "content": user_content}]).choices[0].message.
    content
    #Ask Another LLM to judge the answer
    return response == "CORRECT"
    #Convert judge response into TRUE/FALSE
```

- Flow



6. CREATE CONCISION EVALUATOR & DEFINE CHATBOT INSTRU

In[6]:

```
def concision(outputs: dict, reference_outputs: dict)
 -> bool:
    return int(len(outputs["response"]) < 2
    (reference_outputs["answer"]))
    # check whether response is less than
    # twice the reference answer's length
default_instructions = """
Respond to the user's question in a short, concise
manner (one short sentence). """
#Tell the chatbot to produce short answers
```

7. BUILD CHATBOT & CREATE LANG-SMITH TARGET FUNCTION

In[7]:

```
def my_app(question:str, model:str="gpt-4o-mini",
    instructions:str = default_instructions) -> str:
    response = openai_client.chat.completions.create(
        model=model, temperature=0,
        messages=[{"role": "system",
                   "content": instructions},
                  {"role": "user",
                   "content": question},])
    # send question to the LLM
    return response.choices[0].message.content
    #return only the generated text
def ls_target(input: dict) -> dict:
    question = input["question"] #Extract question from dataset
    response = my_app(question) # Run our chatbot
    return {"response": response}
# Create Langsmith Target Function
```

8. RUN EVALUATION

In[8]:

```
experiment_results = client.evaluate(
    ls_target, #AI Application being tested
    data=dataset_name,#Dataset containing test cases
    evaluators=[correctness, concision],
    #Evaluators to run
    experiment_prefix="openai-4o-mini-chatbot")
    #Name for this experiment
```

We can see evaluation in our langsmith profile we easily check evals of model.

9. COMPARING DIFFERENT MODELS

In[9]:

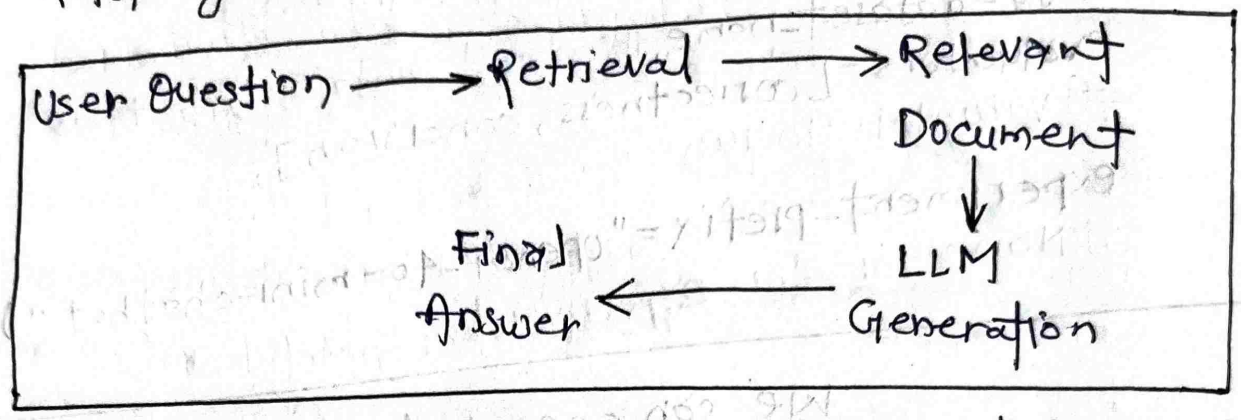
```
def ls_target(inputs:str) ->dict:
#Target function: runs the chatbot with gpt-4-turbo.
    return{
        "response": my_app(inputs["question"],
        model="gpt-4-turbo")}
        #Generate response using the selected model
experiment_results = client.evaluate(ls_target,
#run evaluation on the dataset & target chatbot
    data=dataset_name, #Test dataset
    evaluator=[correctness, concision] #Evaluation Metrics
experiment_prefix="openai-4-turbo-chatbot",)
#Expermient Name
```

* Langsmith Allows us to use the same evaluation dataset & Evaluators to compare different LLMs.

→ RAG Evaluation & custom Evaluators

↳ RAG Evaluation

That's the Basic flow of a RAG system:



* Evaluating only the final answer is not enough!

(A) A RAG system can fail because:

- Wrong documents were retrieved.
- correct documents were retrieved but ignored.
- The answer contains hallucinations
- The answer doesn't address the question
- The answer is factually incorrect.

(B) Four Core RAG Metrics:

Metric	comparison	purpose
correctness	Answer ↔ Reference	Is the answer factually correct?
Relevance	Answer ↔ Question	Does the Answer, answer the Question?
groundness	Answer ↔ Documents	Is the answer supported by retrieved context?
Retrival Relevance	Documents ↔ Question	Did retrieval find useful information?

Note - Each Metric detects a different failure in the RAG pipeline.

↳ Structured Output

- Use TypedDict / Pydantic to define evaluator output
- Usually contains:
  - * explanation → reason for the judgement
  - * correct / relevant / grounded → True or False
- with_structured_output() makes LLM judge responses consistent & machine-readable.

↳ Correctness Evaluator

Check whether the generated answer matches the ground-truth answer.

- Input: Question + Ground Truth + Student Answer
- Output: True/False

↳ Relevance Evaluator

It checks whether the answer actually addresses the user's question.

- Input: Question + Answer

* No Ground truth is required.

↳ Groundness / Faithfulness

It checks whether the answer is supported by retrieved documents & doesn't hallucinate

- Input - Retrieved Documents + Answer
- Output - True/False

↳ Retrieval Relevance

check whether the retriever fetched useful documents for the question.

- Input: Question + Retrieved Documents
- If it fails, we investigate;
  - * chunking
  - * Embeddings
  - * Retrieval parameters
  - * Vector database

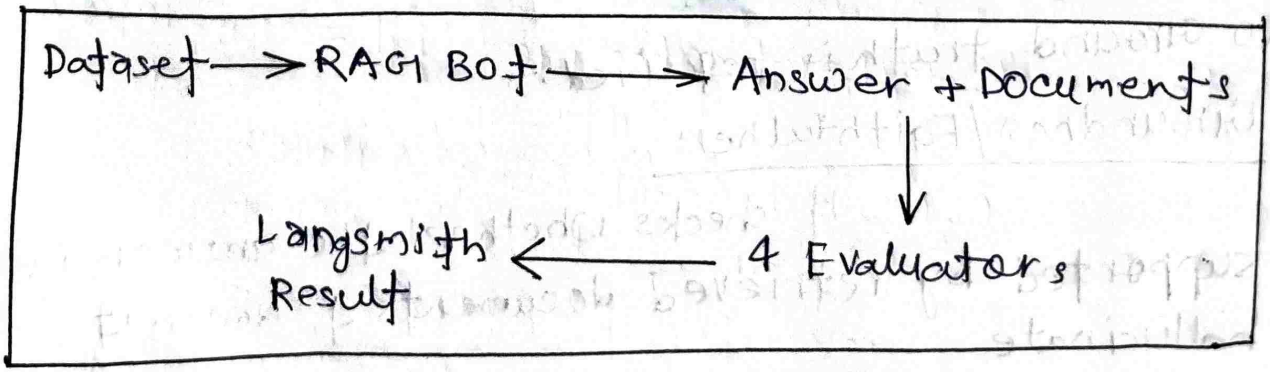
↳ RAG Bot

A rag function should return both.

```
{
    "answer" : answer,
    "documents" : docs
}
```

That's important because groundedness & retrieval relevance need the retrieved documents.

↳ Running Evaluation flow



PROJECT : 1

RAG EVALUATION SYSTEM

Overview

- Build a complete RAG pipeline with document loading, chunking, embeddings & vector retrieval.
- Generates answer using an LLM from retrieved documents
- creates a test dataset with questions & reference answers
- Uses Langsmith for tracing, experiments & evaluation.
- Implements 4 custom LLM-as-Judge metrics
  (1) correctness (3) Groundness
  (2) Relevance (4) Retrieval Relevance
- Analyze evaluation results using a Pandas DataFrame.

InMemoryVectorStore - A Temporary vector database that stores document embeddings in memory for fast similarity-based retrieval.

→ 1. BUILD RAG PIPELINE

In[1]:

```
from langchain_community.document_loader import
WebBaseLoader
from langchain_core.vectorstores import
InMemoryVectorStore
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import
RecursiveCharacterTextSplitter
url = ["Give url links of documents or data",
       "we can add multiple link"]
#source documents
docs = [WebBaseLoader(url).load() for url in urls]
#Load documents from URLs
docs_list = [item for sublist in docs for item
             in sublist]
#Flatten the documents lists
text_splitter = RecursiveCharacterTextSplitter.from_
tiktoken_encoder(chunk_size=250, chunk_overlap=0)
#split documents into smaller chunks
doc_split = text_splitter.split_documents(docs_list)
# create document chunks
vectorstore = InMemoryVectorStore.from_documents(
    documents = doc_splits,
    embedding = OpenAIEmbeddings(),)
# store chunks as embeddings in vector store
retriever = vectorstore.as_retriever(k=6)
# Retrieve top relevant chunks
```

→ 2. CREATE RAG BOT

In[2]:

```
from langchain.chat_models import init_chat_model
from langsmith import traceble
llm = init_chat_model("openai:gpt-4o-mini")
#Initialize the LLM
@traceable()
def rag_bot(question:str) -> dict:
    docs = retriever.invoke(question)
    #Retrieve relevant documents
    docs_string = " ".join(doc.page_content for
    doc in docs)
    #convert retrieved documents into text
    instructions= f""" 
    you are a helful assistant who analyzes
    source information & answers questions.
    #Add More Prompt
    Documents :{docs_string} """
    ai_msg = llm.invoke([
        {"role":"system","content":instructions},
        {"role": "user","content":question},])
    #Generate the final answer
    return{
        "answer": ai_msg.content,
        "documents": docs}
```

3. CREATE EVALUATION DATASET

In [3]:

```
from langsmith import client
client = Client
#Connect to Langsmith
examples = [ { "inputs": { "Question": How does the
              ReAct agent use self-reflection?",
             "output": {          #Add More
                 "answer": (- --) } ---- y, y, ]
# Test Question + reference answers.
dataset_name = "RAG Test Evaluation"
dataset = client.create_dataset(dataset_name =
 dataset_name)
#create evaluation dataset
client.create_examples (dataset_id = dataset.id,
    examples = examples )
#Add test example to dataset
```

4. CUSTOM EVALUATORS, EVALUATOR-1: CORRECTNESS

In [4]:

```
from typing_extensions import Annotated, TypedDict
from langchain_openai import ChatOpenAI
class correctnessGrade (TypedDict):
    explanation : Annotated [str,
        "Explain the reasoning for the score"]
    correct: Annotated [bool,
        "True if the answer is factually correct"]
#Defines structured output of the judge
```

```
correctness_instructions = """
    #give prompt/Instruction you are a teacher
      grading a quiz. """
# Instructions for correctness judge
grader_llm = ChatOpenAI(model="gpt-4o-mini",
    temperature=0).with_structured_output(
    CorrectnessGrade, method="json_schema",
    strict=True)
#LLM used as the correctness judge
def correctness(inputs: dict, outputs: dict,
    reference_outputs: dict) -> bool:
    answer = f""
    QUESTION: {inputs['question']}
    GROUND TRUTH: {reference_outputs['answer']}
     """
    # Prepare question, reference & generated answer
    grade = grader_llm.invoke([
            { "role": "system"
              "content": correctness_instructions},
            { "role": "user"
              "content": answers},  ])
    #Ask LLM judge to evaluate correctness
    return grade["correct"]
    #Return True/False score
```

→ 5. EVALUATOR-2 : RELEVANCE

In[5]:

```
class RelevanceGrade(TypedDict):
    explanation: Annotated[str,
     "Explain the reasoning for the score"]
     relevant: Annotated[bool,
     "True if the answer addresses the question"]

relevance_instructions = """
    You are a teacher grading a quiz
    #give a instruction/prompt """
#Instructions for relevance judge
relevance_llm = ChatOpenAI(model="gpt-4o",
   temperature=0).with_structured_output(
   RelevanGrade, method="json_schema",
   strict=True)
#LLM used as a relevance judge
def relevance(inputs:dict, outputs:dict) -> bool:
    answer = (f"Question: {inputs['question']}\n"
        f"STUDENT ANSWER: {outputs['answer']}")
    #Compare Question with generated answer
    grade = relevance_llm.invoke([
          {"role": "system",
           "content": relevance_instructions},
          {"role": "user"
           "content": answer},])
    # Ask judge to evaluate relevance
    return grade["relevant"]
    # Return True/False score
```

→ 6. EVALUATOR-3: GROUNDEDNESS

In [6]:

```
class GroundedGrade(TypedDict):
    explanation: Annotated[str,
    "Explain the reasoning for the score"]
    grounded: Annotated[bool, "True if the answer is
     supported by the documents"]
grounded_instructions = """
    you are a teacher grading a quiz """
    #Give a detailed prompt/Instruction
#Instructions for groundedness judge
grounded_llm = ChatOpenAI(model="gpt-4o",
    temperature=0).with_structured_output(
    GroundedGrade, method="json_schema",
    strict=True)
#LLM used as groundedness judge
def groundedness(inputs:dict, output:dict) -> bool:
    doc_string = "\n\n".join(doc.page_content for
    doc in outputs["documents"])
    #Convert retrieved documents into text
    answer = (
        f"FACTS: {doc_string}\n"
        f"STUDENT ANSWER: {outputs['answer']}"
    #Compare documents with generated answer
    grade = grounded_llm.invoke([
        {"role": "system",
         "content": grounded_instructions},
        {"role": "user",
         "content": answer}, ])
    #Ask judge to detect unsupported information
    return grade["grounded"]
    #Return True/False score
```

7. EVALUATOR : 4 - RETRIEVAL RELEVANCE

In [7]:

```
class RetrievalRelevanceGrade(TypedDict):
    explanation: Annotated[str,
    "Explain the reasoning for the score"]
    relevant: Annotated[bool,
    "True if retrieved documents are relevant"]

retrieval_relevance_instructions = """
  you are evaluating a RAG retrieval system
  #write a detailed prompt/Instruction
#Instruction for retrieval judge
retrieval_relevance_llm = ChatOpenAI(model="gpt-4o",
    temperature=0).with_structured_output(
    RetrievalRelevanceGrade, method="json_schema",
    strict=True)
#LLM used as retrieval judge

def retrieval_relevance(inputs:dict, outputs:dict)
 -> bool:
    doc_string = "\n\n".join(doc.page_content for
    doc in outputs["documents"]
    # Convert retrieved documents into text
    answer = (
        f"FACTS: {doc_string} \n"
        f"QUESTION: {inputs['question']}")
    #Compare Question with retrieved documents
    grade = retrieval_relevance_llm.invoke([
        {"role": "system",
         "content": retrieval_relevance_instructions},
        {"role": "user",
         "content": answer},])
    return grade["relevant"] #Return True/False score
```

8. RUN COMPLETE RAG EVALUATION

In[8]:

```
def target(inputs: dict) -> dict:
    return rag_bot(inputs["question"])
    #RUN RAG application for each test question
experiment_results = client.evaluate(target,
    data = dataset_name,
    evaluators = [correctness, groundedness, relevance,
    retrieval_relevance,],
    #Evaluate four RAG Dimensions
    experiment_prefix = "rag-doc-relevance",
    #Name of the experiment in Langsmith
    metadata = {"version": "LCEL context, gpt-4-
      0125-preview"},)
    #optional experiment result
experiment_result.to_pandas()
#convert result into a Panda DataFrame
```

EVALS - II

LLM EVALUATION: BENCHMARKING & LEADERBOARD

LLM EVALUATION

(A) Standardized Benchmark

Here pre-defined tests used to measure & compare specific model capabilities.

Ex-

- GSM8K - mathematical reasoning
- MMLU - knowledge across subjects
- SWE-bench - software engineering
- ARC-AGI - abstract reasoning

(B) Custom Evaluation

Here evaluations designed specifically for an application or business use case.

Ex-

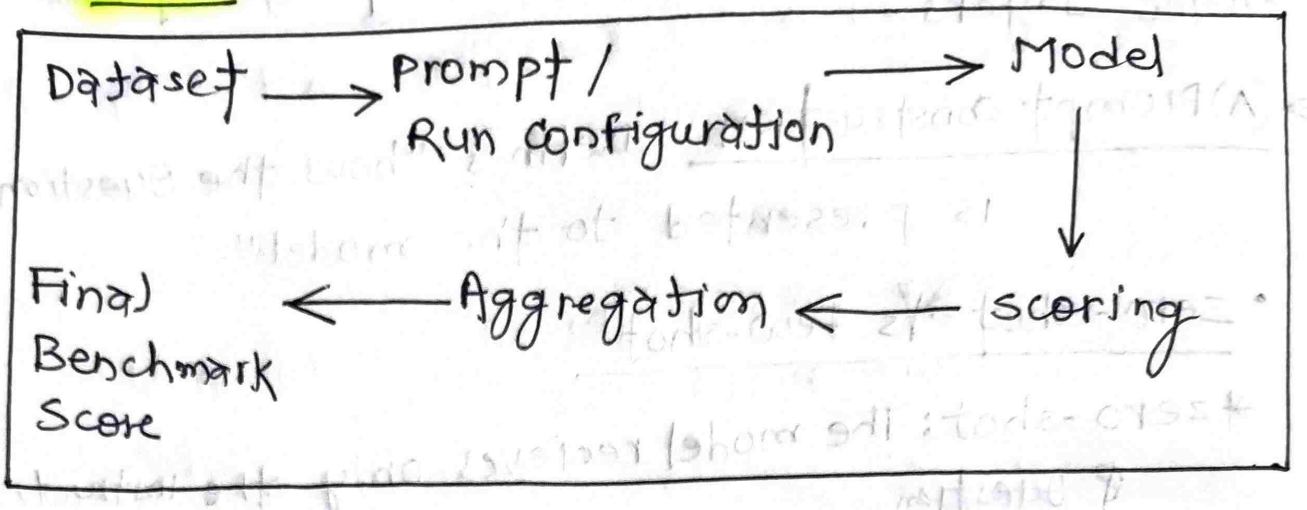
- RAG answer correctness
- Document retrieval quality
- Customer-support response quality
- Agent tool-call accuracy
- Domain-specific factuality

→ **LLM Benchmark**

It is a standardized test designed to measure one or more capabilities of a Language Model under predefined conds.

In short - standardized Exam for an LLM

→ Flow



→ Benchmark's four essential components:

(1.) The Dataset

It contains the evaluation questions / prompts along with their corresponding correct answer keys or expected outputs.

↳ Ex: GSM8K - this benchmark dataset contain grade school mathematical word problems paired with numerical answer.

* "Dataset" - Defines "what the model is being test on".

(2) Run configuration

It defines the operational parameters & enviroment under which model is evaluated

Same Model can produce different benchmark scores depending on "how it is configured".

A Benchmark's run configuration operates across three layers -

↳ **(A) Prompt construction** : Defines "how the Question is presented to the model".

- **zero-shot Vs Few-shot**

* **zero-shot**: The model recieves only the instruction & question.

In short - only question, not any sample with Question + Answer.

* **Few-shot**: The prompt includes example of previous question & correct answers before the actual Question

In short - with few Question + Answer we can set limit also

* It help prompting can help establish the expected reasoning pattern & output format.

- **Chain-of-Thought (CoT) Vs Direct**

* **Chain-of-Thought** - Model is prompted to reason through the problem step-by-step before producing the answer.

* **Direct**: The Model is asked to provide the final answer directly.

↳ (B.) Decoding & Sampling Configuration

of Defines "How the Model generates tokens".

- Temperature - should be 0 to be not be too creative answer.
  - Temperature = 0 → very deterministic
  - " " = 0.2 - 0.7 → low/moderate randomness
  - " " = 1+ → more randomness/creativity
- Max_tokens / output limit - Max amount of output the model can generate.

↳ (C.) Sampling Attempts & Enviroment

Defines "How many times the Model is evaluated & what capabilities / tools it can access".

- Pass@1 : one generation attempt per problem.
- Pass@K : generates "K" attempts for the same problem.
- Majority@K / Self-consistency : generates Multiple Answers & selects the answer receiving the majority vote.
- Tool Availability : such as Python, calculators,

(3) The Scoring Method

Defines "how the model's generated output is converted into a score such as correct/incorrect.

↳ **(A) Extraction**: The system first extracts the target answer from the model's complete output.

Ex- Common extraction strategies-
- Regular Expressions
- Delimiters
- JSON parsing
- structured output parser
- Exact answer fields.

↳ **(B) Comparison** - After extraction, the predicted answer is compared with the ground truth answer.

- **closed-Ended Tasks** - Task with deterministic (fixed) Answer

Ex - Predicted → 72, Ground truth → 72, Result = correct

Like - usually evaluated using
- * Exact Match
- * Numeric comparison
- * Classification Accuracy
- * Structured field Matching

- **Open-Ended Tasks** - where multiple answers can be considered valid.

Ex- "summarize this Article".

* **Evaluation May require:**
- Human evaluation
- LLM-as-as-judge
- Rubric based grading
- Reference based metrics

(4) The Aggregation

It is the process of combining individual evaluation results into an overall benchmark score.

* The Aggregation method Determines how individual score are combined.

↳ **(A) UnWeighted Mean** - Every Evaluation item recieve equal Importance

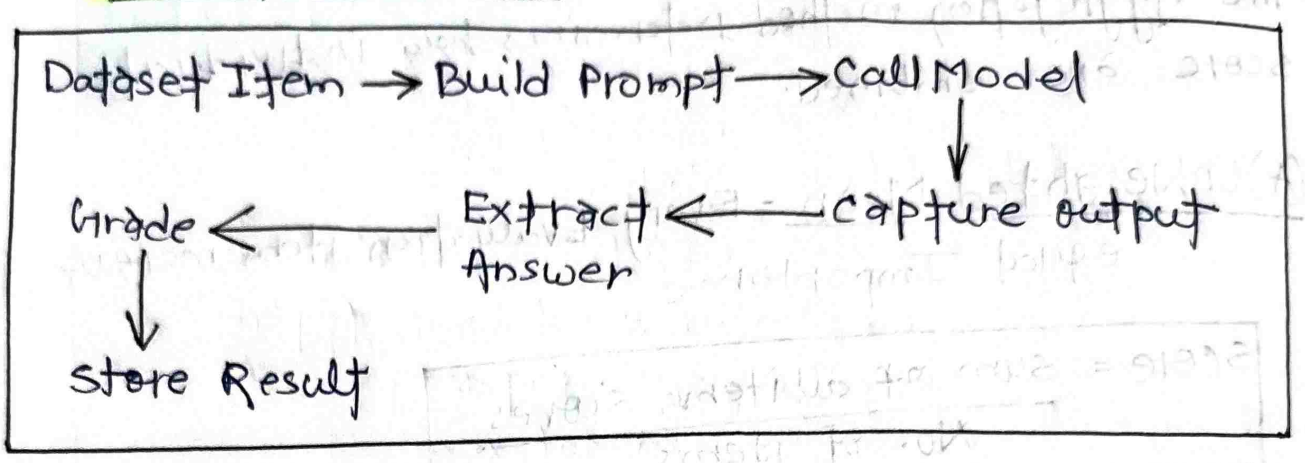
$$\text{Score} = \frac{\text{sum of all items scores}}{\text{No. of items}}$$

↳ **(B) Weighted Mean** - Different categories or groups receive different weights according to the benchmark's defined evaluation protocol.

→ Model Evaluation Using Benchmark

"An Evaluation is a loop over the dataset".

- **Evaluation Pipeline**-



↳ (1.) Load Dataset item

For each evaluation item, load:

- Question/Prompt
- Ground-Truth Answer

↳ (2.) Build the Prompt

Construct the exact prompt that will be sent to the model.

Prompt construction may include:

- Few-shot examples
- system instructions
- User Question
- Task Instructions
- chat Template
- output-format instructions

↳ (3.) Call the Model

send the constructed prompt to the Model using a pinned decoding configuration.

Important parameter Include-

- Temperature
- max Tokens
- other sampling parameter
- Model/Version
- Tool Configuration, if applicable

↳ (4.) Capture the Raw output

store the Model's raw output exactly as generated.

* Raw output are essential for debugging evaluation failures.

↳ (5.) Extract the Answer

The evaluator extracts the actual answer from the model's complete output.

↳ (6.) Grade the Prediction

Compare the extracted prediction with the ground-truth answer.

↳ (7.) Store the Per-Item Result

store the information required to reproduce & debug the evaluation

↳ (8.) Aggregate the Results

After evaluating every item in the dataset, aggregate the individual scores into the final benchmark metric

↳ Evaluation code structure

```
results = [ ]
for item in dataset:
    prompt = build-prompt(
        item,
        fewshot_examples = 8,
        chat_template = chat_template)
    #Build the exact prompt
    output = model.generate(prompt,
        temperature = 0,
        max_tokens = 50)
    #call the model using fixed evaluation setting
    pred = extract_answer(output)
    #Extract the predicted answer
    score = int(pred == item.gold)
    #Compare prediction with ground truth
    result.append((prompt, output, pred, item.gold,
      score))
    #store everything needed for debugging
    accuracy = mean[r[4] for r in results)
    #Aggregate per-item scores
```

→ Evaluation Harness

It is a ready-made system or framework that automates the repeated tasks required to evaluate models against benchmarks.

It handles the evaluation pipeline instead of requiring you to manually build everything.

"Harness typically knows".

(1) Where to get the benchmark data
(2) How to format each question
(3) How to send the question to the model
(4) How to extract the answer
(5) How to judge correctness
(6) How to calculate the final score.

↳ Problems in Manual Evaluation

An evaluator needs to handle.

- Answer Extraction
- Benchmark specific scoring : each benchmark can have its own:
  - Answer format
  - Scoring rules
  - Evaluation metrics
  - Aggregation method
- Large-scale Evaluation
- API failures
- Rate Limits

Benchmark Vs Evaluation Harness

Feature	Benchmark	Evaluation Harness
Meaning	standardized test	system that runs the test
Purpose	"What to test"	"How to test"
contains	Dataset + scoring rules	Prompt + Model calls + scoring
Examples	GSM8K, MMLU	LM Evaluation Harness
Execution	Defines evaluation	Run evaluation
Infrastructure	Usually not responsible for execution	Handles batching, retries, rate limits, concurrency
scale	Evaluation items	Batching, retries, concurrency
output	Benchmark score	Results + logs
Analogy	Exam paper	Exam administration system

PROJECT - 1
GSM8K LLM BENCHMARK

→ Overview

- **Goal** - Evaluate an LLM's Mathematical reasoning ability using GSM8K.
- **Framework** - lm-eval (LM Evaluation Harness)
- **Model** - GPT-5.6 Luna through openAI chat completions API.
- **Evaluation** - 8-shot chain-of-thought (gsm8k_cot).
- **sample** - Tests 20 GSM8K questions.
- **Reliability** - Uses concurrency & automatic retries for API requests.
- **Output** - saves benchmark results & individual sample logs.

→ code

```
!pip install -q "lm_eval[api]"
#Install LM evaluation Harness with API support
import os
from getpass import getpass
os.environ["OPENAI_API_KEY"]=getpass("enter openAI
                                    API Key:")
!lm-eval run\
    --model openai-chat-completions \ # use openAI
    --model_args "model=gpt-5.6-luna,num_concurrent=5,
                                        chat completions
    max_retries=5" \#Model + API Settings
    --tasks gsm8k_cot \# GSM8K chain-of-thought task
    --num_fewshot 8 \ #Provides example before each question
    --apply_chat_template \ #Apply the Model's chat format
    --limit 20 \  #Evaluate only 20 samples
    --output_path ./gsm8k_result \ # save evaluation results
    --log_samples  # save individual sample outputs
```

→ Who runs Model Evals ?

↳ (1.) Frontier Labs

Like - OpenAI, Antropic, Google Deepmind, Meta, Mistral, Deepseek, Qwen etc.

Reason of Frontier Lab Run Model Evals

- Internal Development → guide model improvement
- Release gating → decide whether a model is ready to ship.
- Marketing → publish benchmark result.

↳ (2.) Third Party Evaluators

Like - LM Arena, Artificial Analysis, Epoch AI, Vals AI etc.

Reason

- Independently evaluate models.
- Compare multiple model under defined cond^{n}s
- Measure factors such as Quality, Cost & latency

↳ (3.) AI Engineering Teams

like - companies building AI products.

Reason

- Test whether a model works for their specific application
- Measure Accuracy, latency, cost, reliability & failure case.

→ Why Benchmarks can be Misleading ?

↳ (1.) Benchmark contamination

Benchmark data may appear in a model's training data, potentially making the score less representative of generalization.

↳ (2.) Benchmark saturation

Models may score close to the maxm, making it difficult to distinguish their capabilities.

↳ (3.) Configuration Gaming

small changes in:

- Prompting
- Few-shot examples
- Sampling
- Tools
- Evaluation settings.

can significantly affect results.

↳ (4.) Aggregation

A single overall score can hide weaknesses in specific categories or tasks.

→ LLM Leaderboards

It is public ranking or comparison table showing how different LLMs perform across common evaluation.

↳ Why Leaderboards Exist ?

(1) Compare Models - Provides a common reference for comparing models from different labs.

(2) Save Evaluation Resources - Helps developer to shortlist models instead of evaluating hundred themselves.

(3) Detect Benchmark Saturation - If Models cluster around very high scores, the benchmark may no longer differentiate them well.

(4) Model Discovery - Helps discover smaller, open-weight or less known models

↳ Who Uses Leaderboards

(1) AI Engineers

(2) Frontier Labs

(3) Researchers

(4) Policymakers & safety institutes

(5) open-source/smaller Labs

↳ Types of LLM Leaderboards

(1) Benchmark-specific - Rank model on one benchmark.

Ex- • MMLU
• GSM8K
• HumanEval
• GPQA

Limitation - Narrow view of capability.

(2) Multi-Benchmark - Combines multiple evaluation.

May Measure:

- Knowledge
- Reasoning
- Mathematical
- Coding
- Cost
- Latency
- Throughput

(3) Human-Preference Based - Uses human comparison bet^{n} model responses.

Ex- Model A Response ⟷ Model B Response
↓
Human Vote

Measures:

- Helpfulness
- clarity
- writing
- creativity
- conversational quality.

(4) Application specific - Focuses on a particular real-work tasks

Ex- • coding
• Tool/function calling
• SQL generation
• Medical QA
• Agentic Task
• Long-document processing.

↳ Why can't we Blindly trust Leaderboards

(1) Poor Real-World Transfer

Benchmark are often clean & self contained

Real Application contain:

- Ambiguous requests
- Missing information
- Company-specific data
- Multi turn conversations
- Tool failures
- Edge cases

(2) Benchmark Contamination

Benchmark questions may appear in training data.

High performance may therefore come from:

capability + Memorization + Familiarity

3) Leaderboard overfitting

Developers may repeatedly optimize models specifically for a leaderboard.

ote:- Godfather's Law — When a measure become a target, it can become less useful as a measure

4) Small score difference

Suppose - Model A = 84.3
Model B = 84.1

The difference may come from:

- Evaluation noise
- small test sets
- Prompt differences
- Grading errors

Small rank difference may not represent meaningful capability differences

5) Aggregation choices

overall ranking depend on choices such as:

- which benchmarks are included
- score normalization
- Benchmark weights
- cost/latency weights

changing these choices can change ranking.

(6) Human Preference Bias

people may prefer answers that are:

- Longer
- More confident
- Better formatted
- More agreeable
- More entertaining

Preference does not always equal factual accuracy.

(7) Stale or self-Reported Results

Leaderboards may contain

- older model versions
- Missing new models
- Discontinued models
- Unverified vendor-reported results

→ How to Read Leaderboards as an AI engineer

(1) Define Requirements first

write down:

- Task Type
- Cost Limit
- Latency requirement
- context requirements
- Deployment constraints

(2) choose the Right Leaderboard

Match the leaderboard to ours task:

- Agents → Agent benchmarks
- Tooluse → Function-calling benchmarks
- RAG → RAG benchmarks
- chat → Human-preference benchmarks
- Budget → cost + throughput metrics

(3) check the Methodology

Ask:

- what exactly is being scored?
- who ran the evaluation,
- what inference settings were used?
- How old is the dataset,
- Is the benchmark saturated,
- Are confidence intervals provided?
- How are composite scores weights?

(4) Shortlist 3-5 Models

Apply hard constraints first:

License → Price → Latency → Availability

(5) Run our own Evaluation

Use real or represent

examples & measure:

Quality + cost + Latency + Failure Behaviour